

Assignment: Centralized Logging System using Golang Microservices and Docker

Objective

Design and implement a centralized logging system using Golang microservices, containerized with Docker.

The system should simulate multiple client microservices that generate logs and send them to a **log collector microservice**, which then forwards and stores the logs in a **central logging microservice**.

A user-facing API should allow querying the logs.

This assignment assesses your understanding of:

- Microservice communication
 - Go concurrency
 - API design
 - Docker orchestration
-

Architecture Overview

The system includes the following components:

1. Client Microservices

You must implement microservices that simulate system-level logs for either **Linux** or **Windows** environments. These services act as log clients that generate log data and send it to the **Log Collector** service via TCP or UDP.

Option A: Linux Log Simulation

Choose **any two** of the following categories and simulate corresponding logs:

- Syslog (e.g., kernel, cron, auth)
- Login Audit (user login attempts with success/failure)
- Logout Audit (session terminations)

Example log format

```
{"message": "<86> aiops9242 sudo: pam_unix(sudo:session): session opened for user root(uid=0) by motadata(uid=1000)"}
```

Option B: Windows Log Simulation

Choose any two of the following categories and simulate corresponding logs:

- Windows Login Audit (successful/failed logins)
- Logout Audit (user sessions ending)
- Event Logs (application/system events)
- File Audit (file read/write/delete access)
- User Audit (user creation, privilege changes)

Example log format

```
{"message": "<134> WIN-EQ5V3RA5F7H Microsoft-Windows-Security-Auditing: A user account was successfully logged on. Account Name: Motadata"}
```

Behavior & Flow

Each client service should:

- Periodically (every 1–2 seconds) generate logs
- Send logs to the log collector via **TCP or UDP**
- Transmit logs in **JSON format**

Example Message Structure

```
{"timestamp": "2025-07-29T12:35:24Z", "hostname": "aiops9242", "event.source.type": "linux", "event.category": "login.audit", "message": "<86> aiops9242 sudo: pam_unix(sudo:session): session opened for user root(uid=0) by motadata(uid=1000)"}
```

Expectations

- Each client microservice runs independently.
- Logs are structured and realistic.
- Network transport is correctly configured (TCP/UDP).

2. Log Collector Microservice (log-collector)

- Accepts logs from clients via a **TCP/UDP listener**
- **Validates the payload** and **parses the message** to extract fields such as **"timestamp"**, **"event.source.type"**, **"severity"**, **"hostname"**, **"event.source"**, **"event.category"**, and **"username"**, based on the log category

- **Enriches parsed fields** by checking against a list of blacklisted IPs or usernames (e.g., adds "is.blacklisted": true if matched)
- **Forwards valid logs** to the Central Logging Microservice via POST /ingest
- Supports **concurrent log handling** using **goroutines, channels, or worker pools** for scalable processing

This microservice acts as a **receiver** for logs coming from multiple client microservices over **TCP or UDP**, processes them, and forwards valid logs to a **Central Logging Microservice**.

What it should do

1. Listen for Logs

- Accept incoming logs over **TCP or UDP** (choose any one).
- Receive logs in JSON format with the following structure: { "message": "<86> aiops9242 sudo: pam_unix(sudo:session): session opened for user root(uid=0) by motadata(uid=1000)"} }

2. Parse Logs

- Extract basic fields from the message string such as:
 - timestamp (current time or from log if available)
 - "event.category" (e.g., login.audit, logout.audit)
 - "event.source.type" (e.g., Linux, Windows)
 - "username" (e.g., root or motadata)
 - "hostname" (e.g., hostname aiops9242)
 - "severity" (from the syslog code like <86>)
- You can extract these using simple string parsing or regex.
- These are the reference log formats; the Log Collector should be capable of parsing and extracting fields based on the raw message.

3. Enrich Logs

- If the log contains a blacklisted IP or username (use a small, hardcoded list), add a field: "is.blacklisted": true

4. Forward to Central Logging Service

- Send the parsed and enriched log to: POST /ingest

- Payload example: { "timestamp": "2025-07-29T12:35:24Z",
"event.category": "login.audit", "username": "root", "hostname":
"aiops9242", "severity": "INFO", "raw.message": "<86> aiops9242 sudo:
pam_unix(sudo:session): session opened for user root(uid=0)",
"is.blacklisted": false}

5. Handle Logs Concurrently

- Use **goroutines and channels** so multiple logs can be processed at once.
- Each incoming log should be processed and forwarded without blocking others.

Key Expectations

- Accept logs over TCP or UDP
 - Parse the message to extract basic fields
 - Enrich if blacklisted user/IP is found
 - Forward logs via HTTP to central logging service
 - Handle multiple logs at once using concurrency
-

3. Central Logging Microservice (log-server)

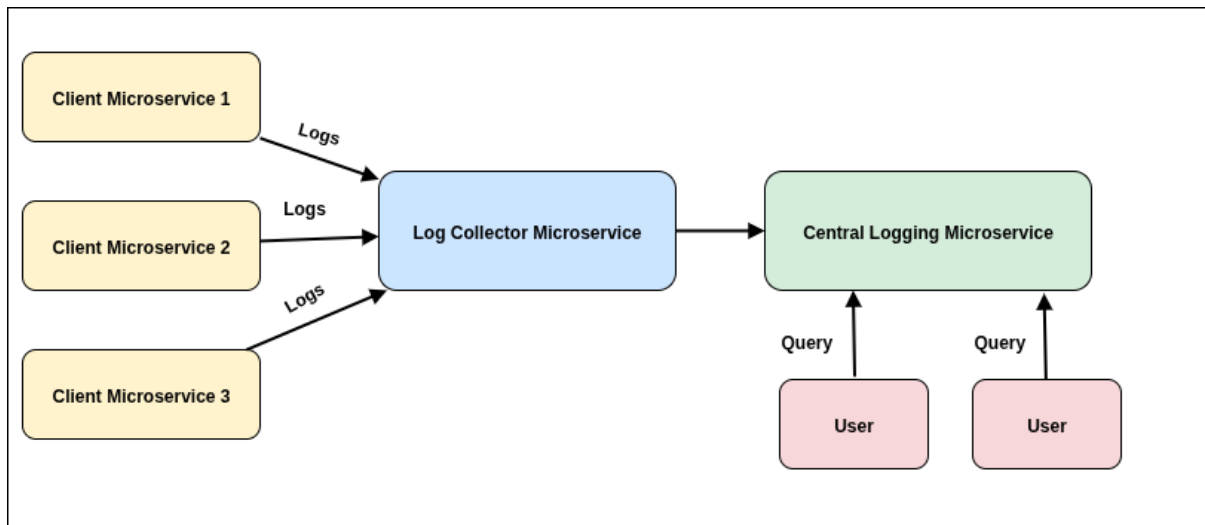
- Receives logs via POST /ingest
 - Stores logs (File-based structures or NoSQL Databases) in logging microservices
 - Provides a query API at GET /logs with filtering support
-

4. Supported Queries by Central Logging

Allow the following query types via query parameters:

- **GET /logs?service=windows_logout**
- **GET /logs?level=error**
- **GET /logs?service=linux_login&level=warn**
- **GET /logs?username=root & "is.blacklisted"=true**
- **support for:**
 - **limit (e.g. ?limit=10)**

- **sort (e.g. ?sort=timestamp)**



Dockerization

- Dockerize each microservice (client, log-collector, log-server)
- Use docker-compose.yml to launch all components on the same network
- All services should be independently deployable
- Ensure logs can be forwarded and queried within Docker containers

Additional Requirements

- Add a /metrics endpoint to show:
 - Total logs received
 - Logs grouped by event.category and severity
- Include **unit tests** for critical components (validation, storage, routing)
- Abstract log storage using **Go interface**

Submission Requirements

- Submit a public GitHub or GitLab repo
- The repo must include:
 - All Go source code

- Dockerfile(s) for each microservice
 - A working docker-compose.yml
 - README with:
 - Setup instructions
 - API usage examples (curl/Postman)
-

Evaluation Criteria

- Effective use of Go concurrency and features
- Clean and modular API design
- Docker orchestration and service communication
- Code readability and maintainability
- Fulfillment of all core and additional requirements